Session-Scroll-Bug

Root-Cause-Analyse & Fix-Plan

AgenticExplorer v1.6.26-dev · 2026-04-23

Problem-Owner: hossoOG
Analyse-Team: 4 parallele Explore-Subagenten (D1-D4)
Release-Gate: Fix vor v1.6.26-Push auf master

Analyse-Dokument `reports/session-scroll-bug-analyse-v1.6.26.md`

TL;DR — Drei Sätze

1. **Bei jedem Session-Tab-Switch wird das xterm.js Terminal komplett disposed und beim Zurücktabben als leeres neues Terminal wiedergeboren.**
2. **Der gesamte Scrollback-Verlauf geht verloren** — `scrollback: 5000` Zeilen spielt keine Rolle, weil der Buffer mit `term.dispose()` zerstört wird.
3. **Fix: Terminal-Cache** `Map<sessionId, Terminal>` **+ Output-Ring-Buffer im Store.** ~80 LOC, 1 Regression-Test, minimal invasiv.

Das Symptom

> **„Das Scrollen in den Sessions ist leider immernoch verbuggt.
> Nicht mal mehr das resizen trick was ich mach funktioniert
> weil einfach Daten fehlen ich kann ein stück hochskrollen
> ABER es fehlt verlauf.... Das nervt mich ENORM"**

Übersetzt:

- Scrollen selbst funktioniert (Viewport bewegt sich hoch/runter)
- Aber der Scrollback-Puffer hat **weniger Content als er sollte**
- Frühere Bytes sind **echt weg**, nicht nur versteckt
- Der bekannte „Resize-Trick" (Fenster ziehen → Rerender) hilft nicht mehr, **weil die Daten tatsächlich nicht mehr im Buffer liegen**

Warum das enorm nervt

Aspekt	Impact
Frequency	Bei jedem Tab-Switch zwischen Sessions — also dutzendfach pro Arbeitssitzung
Recovery	Kein Workaround mehr (Resize-Trick ist tot)
Datennot	Claude-Agent-Runs mit 5000–50000 Zeilen Output sind keine Seltenheit → langes Debug-Readback unmöglich
Mental Load	User muss sich merken, was er gelesen hat, weil er's nicht mehr nachlesen kann
Trust	Eine Session-Manager-App die Session-Output „vergisst" ist ein **Vertrauensbruch** auf der Produktvision

Das ist kein kosmetischer Bug. Das torpediert die Kern-Value-Proposition.

Architektur-Überblick

```
┌──────────────────────────────────────────────────────────┐
│ AgenticExplorer (Tauri v2 Desktop-App)                   │
├──────────────────────────┬───────────────────────────────┤
│  Backend (Rust)          │  Frontend (React+TS)          │
├──────────────────────────┼───────────────────────────────┤
│ src-tauri/               │ src/                          │
│ ├ session/               │ ├ components/sessions/        │
│ │  ├ manager.rs          │ │  ├ SessionTerminal.tsx      │
│ │  └ commands.rs         │ │  └ hooks/                   │
│ │                        │ │     └ useSessionEvents.ts   │
│ └ (portable-pty)         │ ├ store/sessionStore.ts       │
│                          │ └ (@xterm/xterm 6.0)          │
└──────────────────────────┴───────────────────────────────┘
        │                                  ▲
        │   Tauri IPC Event                │
        └──────  "session-output"──────────┘
             { id, data: bytes }
```

Bytes-Flow: PTY → Rust-Reader-Thread → IPC-Event → Global-JS-Hook → SessionTerminal → xterm.js

Die 6 Stages der Output-Pipeline

#	Stage	File	Rolle
1	PTY Reader	`src-tauri/src/session/manager.rs`	Liest Bytes vom Child-Prozess in 4KB-Chunks
2	IPC Emit	`manager.rs:193`	`emit("session-output", {id, data})`
3	Global Hook	`useSessionEvents.ts`	Einmaliger Listener, Rolling-500B-Preview-Buffer
4	Store	`sessionStore.ts`	**Nur `lastOutputSnippet` (200 Bytes!)**
5	Component	`SessionTerminal.tsx`	Erzeugt xterm, registriert **zweiten** Listener
6	xterm.js	`@xterm/xterm 6.0`	Verwaltet Scrollback im RAM

Der Bruch passiert zwischen Stage 5 und 6 — beim Tab-Switch.

Stage 1 — PTY Reader (Rust)

`src-tauri/src/session/manager.rs:177-294`

```rust
// Reader Thread (spawned per Session)
let reader_thread = thread::spawn(move || {
    let mut buf = [0u8; 4096];                              // 4KB fixer Chunk
    loop {
        match reader.read(&mut buf) {
            Ok(0) => break,                                 // EOF
            Ok(n) => {
                let data = String::from_utf8_lossy(&buf[..n]).to_string();
                read_app.emit("session-output",
                    SessionOutputPayload { id: &id, data: &data }
                ).unwrap_or_else(|e| log::debug!( ... ));    // Fire-and-forget
            }
            Err(e) => { log::warn!("reader error: {e}"); break; }
        }
    }
});
```

Findings (D1):

- Kein Rust-side Output-Buffer / History-Storage

Stage 2 — IPC "session-output" Event

Payload: `{ id: string, data: string }`

```
// Beispiel eines Events:
{
  id: "session-1776874109183-nerppq",
  data: "Running tests ... \r\n  PASS  src/foo.test.ts (4 tests)\r\n"
}
```

Eigenschaften:

- Asynchron, fire-and-forget
- Keine Garantie der Auslieferung bei unmountetem Listener
- **Kein Replay** wenn Frontend zu spät subscribed
- Tauri queued intern, aber flusht bei voller Queue

Stage 3 — useSessionEvents (Global Hook)

`src/components/sessions/hooks/useSessionEvents.ts:34-64`

```
const outputBuffers = useRef<Map<string, string>>(new Map());

listen<{ id: string; data: string }>("session-output", (event) => {
  const { id, data } = event.payload;
  let currentBuf = buffers.get(id) || "";
  currentBuf += data;
  if (currentBuf.length > 500) currentBuf = currentBuf.slice(-500); // ⚠️ TRIM
  buffers.set(id, currentBuf);

  // Debounced (300ms): nur .lastOutputSnippet (200 chars) in Store
  setTimeout(() => {
    useSessionStore.getState().updateLastOutput(id, buffers.get(id)?.slice(-200));
  }, 300);
});
```

Der Rolling 500-Byte-Buffer ist NUR für die Session-Card-Preview.

Die echten Bytes werden hier **nie archiviert**. Sie gehen direkt weiter zu einem **zweiten** Listener in SessionTerminal.

Stage 4 — SessionStore (was fehlt)

`src/store/sessionStore.ts:20–32`

```
export interface ClaudeSession {
  id: string;
  title: string;
  shellKind: ShellKind;
  folder: string;
  status: "starting" | "running" | "waiting" | "done" | "error";
  lastOutputSnippet: string;    // ⚠️ Nur 200 chars für Status-Detection
  lastOutputAt: number;
  createdAt: number;
  // ... kein fullOutput, kein outputHistory, kein ringBuffer
}
```

Der Store hat keine Output-History pro Session.

- Beim Tab-Switch / Remount kann **nichts nachgespielt** werden
- `updateLastOutput` schreibt nur die 200-Zeichen-Preview
- Es gibt keine „Full-Text"-Feld, keine Ring-Struktur, keinen Log

Konsequenz: Sobald xterm is den Buffer verliert, ist der Verlauf **unwiederbringlich**

Stage 5 — SessionTerminal Component

`src/components/sessions/SessionTerminal.tsx:59-184`

```
useEffect(() => {
  if (!containerRef.current) return;

  const term = new Terminal({                    // ⚠️ NEUE INSTANZ pro Effect-Run
    cursorBlink: true,
    scrollback: 5000,                            // 5000 Zeilen im RAM
     ...
  });
  term.open(containerRef.current);

  listen("session-output", (event) => {          // ⚠️ ZWEITER Listener (pro Mount)
    if (event?.payload?.id !== sessionId) return;
    term.write(event.payload.data);
  });

  return () => {
    term.dispose();                              // ⚠️ BUFFER WEG BEI CLEANUP
    terminalRef.current = null;
  };
}, [sessionId, isAtBottom]);                     // ⚠️ sessionId als Dependency
```

Stage 6 — xterm.js Terminal Instance

Config in diesem Repo:

- `scrollback: 5000` Zeilen
- `allowProposedApi: true`
- FitAddon für Resize-Anpassung
- WebLinksAddon für Link-Detection

xterm.js-Semantik:

- Scrollback lebt im **RAM der Terminal-Instanz**
- `term.dispose()` zerstört den kompletten Buffer unwiederbringlich
- Kein automatisches Persist, kein Serialize, kein Restore

Was das für uns bedeutet:

Wenn wir `term.dispose()` rufen, sind die 5000 Zeilen **weg**. Egal ob User zurück zum gleichen `sessionId` tabbt —

Lifecycle: Erster Mount (funktioniert)

```
User öffnet Session A
  │
  ▼
SessionTerminal mount  (sessionId="A")
  │
  ├─ new Terminal({ scrollback: 5000 })      ← leer, OK
  ├─ term.open(container)
  ├─ listen("session-output", handler_A)     ← subscribed
  │
  ▼
PTY schreibt Output
  │
  ├─ emit("session-output", { id:"A", data:"…" })
  ├─ handler_A empfängt Event
  ├─ if id === "A" → term.write(data)
  │
  ▼
Terminal füllt sich, User sieht Output ✅
```

Lifecycle: Tab-Switch (DER BUG)

```
User tabt von A nach B
  │
  ▼
SessionTerminal unmount (sessionId="A")
  │
  ├─ CLEANUP läuft:
  │   ├─ unlisten() für handler_A          ← Listener weg
  │   ├─ term.dispose()                    ← 💥 SCROLLBACK WEG
  │   └─ terminalRef.current = null
  │
  ▼  (währenddessen…)
PTY-A schreibt weitere 50 Zeilen Output
  │
  ├─ emit("session-output", { id:"A", … })
  ├─ useSessionEvents global-handler: outputBuffers["A"] += data  (→ 500B-trim)
  ├─ updateLastOutput: nur Snippet (200 chars) im Store
  │                                         ← ⚠️ Bytes VERLOREN außer 200 chars
  ▼
User tabt zurück zu A
  │
  ▼
SessionTerminal mount (sessionId="A")
  │
  ├─ new Terminal({ scrollback: 5000 })      ← wieder leer
  ├─ listen("session-output", handler_A2)    ← subscribed
  │
  ▼
```

Der Hot-Spot

`src/components/sessions/SessionTerminal.tsx:184`

```
}, [sessionId, isAtBottom]);
```

Diese eine Zeile entscheidet den Lifecycle.

- `sessionId` in den Dependencies → **Effect-Re-Run bei Session-Switch**
- Effect-Re-Run = alter Cleanup läuft = `term.dispose()` = Buffer weg
- Auch `isAtBottom` ist verdächtig — es ist in `useCallback` gewrapped, aber jede Referenz-Änderung triggert ebenfalls einen Re-Run

Konsequenz: `SessionTerminal` behandelt jede `sessionId` als eine **neue Welt** — und zerstört die vorherige.

Cleanup-Code: `term.dispose()`

`src/components/sessions/SessionTerminal.tsx:174-183`

```
return () => {
  clearTimeout(initialTimer);
  clearTimeout(scrollTrackTimer);
  debouncedFit.cancel();
  unlistenPromise.then((unlisten) => unlisten()).catch(() => {});
  scrollDisposable.dispose();
  resizeObserver.disconnect();
  term.dispose();                          // 💥 Das Kernproblem
  terminalRef.current = null;
};
```

`term.dispose()` laut xterm.js-Doku:

> *„Disposes of the terminal. This frees all DOM elements, event listeners, and internal state. After calling this, the terminal cannot be used anymore."*

Alles futsch: DOM, Events, **Scrollback**, Cursor-State, Selection.

Hypothesen — Ranked

#	Hypothese	Evidence	Priority
H1	Terminal-Dispose bei Tab-Switch → Scrollback weg	`useEffect [sessionId]` + `term.dispose()` im Cleanup	**PRIMARY**
H2	Scrollback-Limit `5000` zu klein für lange Sessions	`scrollback: 5000` hardcoded	Sekundär
H3	Events während Tab unfokussiert gehen verloren	Listener ist pro-Mount, nicht global-mit-Buffer	Aggravator zu H1
H4	Merge-Konflikt in scroll-Logik (Apr 15)	`13db6d4` Merge-Commit, `isAtBottom()` evtl. unvollständig	Wahrscheinlich falsche Fährte
H5	Kein Rust-Side History-Buffer	`manager.rs` speichert nichts	Struktur-Lücke

H1 + H3 + H5 wirken zusammen. H4 ist vermutlich rot herring.

H1: Terminal-Dispose-and-Recreate — PRIMARY

Aussage: Bei jedem `sessionId`-Wechsel wird xterm.js-Buffer zerstört.

Code-Evidence:

```
// Line 62: Neue Terminal-Instanz bei JEDEM Effect-Run
const term = new Terminal({ scrollback: 5000, ... });

// Line 181: Dispose bei Effect-Cleanup
term.dispose();

// Line 184: Dependency [sessionId] → Effect läuft bei Session-Switch
}, [sessionId, isAtBottom]);
```

Sequenz:

1. User auf Session A → Terminal_1 mit 4200 Zeilen Buffer
2. Tab-Switch auf B → Terminal_1.dispose() → Buffer **gone**
3. Tab-Switch zurück auf A → Terminal_2 mit 0 Zeilen Buffer
4. User scrollt hoch → **findet nur neue Zeilen** (seit Remount)

H2: Scrollback zu klein (Sekundär)

`scrollback: 5000` in `SessionTerminal.tsx:66`.

Für kurze Sessions reichen 5000 Zeilen. Für lange Claude-Agent-Runs (multi-hour Tasks, tausende Tool-Calls) kann das schnell überlaufen — dann wird xterm.js von selbst ältere Zeilen droppen.

Evidence:

- Default xterm.js ist 1000 Zeilen — wir haben 5× erhöht, aber immer noch finite
- Claude mit `--verbose` oder großen Agent-Outputs produziert leicht 10-50k Zeilen

Aber: Auch wenn `scrollback: 50000` → **der H1-Bug überschreibt das.** Selbst 1 Mio. Zeilen helfen nichts, wenn der Buffer bei Tab-Switch eh zerstört wird.

Fix-Kategorie: Quality-of-Life-Verbesserung, nicht Root-Cause.

H3: Event-Loss während Tab unfokussiert

Aussage: Während Session A im Hintergrund ist (Tab zu B gewechselt), kommen `session-output`-Events weiter — aber kein SessionTerminal-Listener ist für A aktiv.

Evidence:

- `useSessionEvents.ts` fängt alle Events global, buffert aber nur 500 Bytes rolling → **komplette Bytes ab Event #N+1 weg**
- `SessionTerminal.tsx` listener ist **pro Mount**, also nur bei aktivem Tab

Konsequenz:
Selbst wenn wir H1 lösen (Terminal-Cache → Buffer überlebt Tab-Switch):
Während der Tab unfokussiert ist, laufen Events ins Leere.

Fix-Kategorie: Aggravator zu H1. Lösung: Listener bleibt global, oder Events landen in Store-Ring-Buffer, der beim Remount nachgespielt wird.

H4: Merge-Konflikt Scroll-Logik — vermutlich nicht

D4's Hypothese: Commit `13db6d4` (15. Apr, Merge) könnte den `isAtBottom()`-Fix aus `9ec3f22` unvollständig übernommen haben → User wird nach 150ms zurück-auto-gescrollt.

Gegenbeweis (Code-Check):

```
// SessionTerminal.tsx:110 (aktueller HEAD)
if (isAtBottom(term)) {
  userScrolledUpRef.current = false;
}
```

Der `isAtBottom`-Check ist da. **Scroll-Lock-Race ist korrekt resolved.**

Wahrscheinlicher Beitrag: Maximal ~150ms Auto-Scroll-Fehler nach Terminal-Mount — aber das würde nur die **jüngsten** Events zurück-scrollen. Der User berichtet **fehlende ältere Bytes**, nicht ein zu-weit-gescrolltes Viewport.

Verdikt: H4 ist nicht die Ursache. Scheidet aus.

H5: Kein Rust-Side History-Buffer

Aussage: Rust-Backend behält keine Kopie der PTY-Bytes. Nach `emit()` sind die Bytes vergessen.

Implikation:

- Selbst wenn Frontend alles perfekt macht: Bei App-Neustart ist die ganze Session-History weg
- `resume_session_id` leitet nur an Claude CLI weiter — kein Output-Replay auf Rust-Seite
- Kein `VecDeque<String>` pro Session auf dem Backend

Fix-Kategorie: Struktur-Lücke, die Defense-in-Depth verhindert.

- Primärfix (H1) macht den Alltag schmerzfrei.
- H5-Fix wäre die Extra-Meile für App-Restart-Robustheit.

Empfehlung: Erst H1+H3 lösen, H5 als Post-v1.6.26-Aufgabe.

Root-Cause-Konsolidierung

Kernursache (H1)

`SessionTerminal` erzeugt bei jedem `sessionId`-Wechsel eine **frische Terminal-Instanz** und **zerstört die alte**. Scrollback ist an die Instanz gebunden → stirbt mit.

Aggravator #1 (H3)

`session-output`-Listener ist Komponenten-lokal. Tabs im Hintergrund bekommen keine Terminal-Writes. Global Buffer speichert nur 500B-Preview — der Rest ist weg.

Aggravator #2 (H5)

Kein Rust-seitiger History-Store. Bei App-Restart bleibt nichts vom Verlauf übrig.

Warum der „Resize-Trick" früher funktioniert hat, jetzt nicht

Resize triggerte `FitAddon.fit()`, was intern Redraw verursachte — wenn der Buffer damals noch existierte, wurde

Strukturelle Lücken im Code

Was fehlt	Wo es fehlt	Konsequenz
Terminal-Instance-Cache	`SessionTerminal.tsx`	Jeder Tab-Switch = neues Terminal
Output-History-Ring-Buffer	`sessionStore.ts`	Kein Remount-Replay
Global persistenter Output-Listener	Separate Store-Action nötig	Events gehen ins Leere
Rust-Side Backlog	`src-tauri/src/session/manager.rs`	App-Restart verliert alles
Session-Serialize-to-Disk (optional)	Neuer `session_history.rs`	Crash-Recovery

Die ersten zwei Zeilen der Tabelle sind das Minimum für v1.6.26.

Fix A — Terminal-Cache

Minimal invasiv, ~40 LOC

```
// Neuer Ref in SessionManagerView (oder eigener Hook):
const terminalCache = useRef<Map<string, Terminal>>(new Map());

// SessionTerminal.tsx useEffect (umgebaut):
useEffect(() => {
  let term = terminalCache.current.get(sessionId);
  if (!term) {
    term = new Terminal({ scrollback: 10000, ... });
    term.loadAddon(new FitAddon());
    term.loadAddon(new WebLinksAddon());
    terminalCache.current.set(sessionId, term);
  }
  if (term.element?.parentElement !== containerRef.current) {
    term.open(containerRef.current);  // attach to current DOM
  }
  fitAddon.fit();
  terminalRef.current = term;

  // Cleanup: NICHT dispose, nur detach
  return () => {
    // keep the instance in cache - no dispose
  };
}, [sessionId]);
```

Fix B — Store-Ring-Buffer

Robustes Fundament, ~80 LOC

```
// sessionStore.ts:
export interface ClaudeSession {
  // ... existing
  outputHistory: string;       // ring buffer, e.g. letzte 200KB
}

updateOutputHistory: (id, data) => {
  const MAX_HISTORY = 200_000;
  const s = get().sessions.find(s => s.id === id);
  if (!s) return;
  let next = (s.outputHistory ?? "") + data;
  if (next.length > MAX_HISTORY) next = next.slice(-MAX_HISTORY);
  // update…
}
```

```
// SessionTerminal.tsx mount-hook:
useEffect(() => {
  // 1) Neue Terminal-Instanz
  // 2) Hydrieren: Store-History auslesen, einmal term.write
  const history = useSessionStore.getState().sessions.find(s => s.id === sessionId)?.outputHistory;
```

Fix C — Kombination (empfohlen)

Terminal-Cache + Store-Hydrate-Fallback

Schicht 1 — Hot Path: Terminal-Cache `Map<sessionId, Terminal>` (Fix A).
→ Tab-Switches sind instantan, kein Rendering-Overhead.

Schicht 2 — Cold Path: Store-Ring-Buffer mit 200KB pro Session (Fix B).
→ Bei App-Reload oder Cache-Evict kann Terminal neu gefüllt werden.

Schicht 3 — Global Listener: `useSessionEvents` schreibt neben dem Preview-Snippet auch in `outputHistory` des Store.
→ Auch während Tab unfokussiert: nichts geht verloren.

Ergebnis: Jeder Byte der via `session-output` kommt, landet im Store-Ring-Buffer. xterm.js-Instanz lebt so lange wie die Session existiert. Bei App-Reload hydratisiert xterm aus Store.

LOC-Schätzung: ~120 gesamt, 2h Implementation, 1h Tests.

Empfohlener Fix — Implementation-Plan

Phase 1 — Store erweitern (~25 LOC)

- `ClaudeSession.outputHistory: string` Feld
- `updateOutputHistory(id, chunk)` Action mit 200KB-Cap
- `clearOutputHistory(id)` Action (für removeSession)

Phase 2 — Global-Hook erweitern (~15 LOC)

- In `useSessionEvents.ts` Handler zusätzlich `updateOutputHistory(id, data)` rufen
- Rolling-500B-Preview bleibt wie bisher für die Session-Card

Phase 3 — Terminal-Cache (~50 LOC)

- Neuer Ref `terminalCacheRef` auf SessionManagerView-Ebene
- `SessionTerminal` liest/schreibt den Cache via Context oder Props
- Cleanup: `removeSession` ruft `term.dispose()` + evict aus Cache

Regression-Test (Vitest)

```
import { renderHook, act } from "@testing-library/react";
import { useSessionStore } from "@/store/sessionStore";

test("outputHistory überlebt Tab-Switch", () => {
  const { result } = renderHook(() => useSessionStore());

  act(() => {
    result.current.createSession({ id: "A", ... });
    result.current.updateOutputHistory("A", "line 1\n");
    result.current.updateOutputHistory("A", "line 2\n");
  });

  // Simuliere Tab-Switch (Mount/Unmount)
  const { rerender } = render(<SessionTerminal sessionId="B" />);
  rerender(<SessionTerminal sessionId="A" />);

  // Assert: History wird zurück in term.write gestreamed
  expect(result.current.getOutputHistory("A"))
    .toBe("line 1\nline 2\n");
});

test("Ring-Buffer capped at 200KB", () => {
  act(() => {
    result.current.updateOutputHistory("A", "x".repeat(250_000));
  });
  expect(result.current.getOutputHistory("A").length)
    .toBe(200_000);
});
```

E2E Playwright Test (ergänzend)

```
// e2e/session-scroll.spec.ts
test("Output-Verlauf überlebt Tab-Switch zwischen Sessions", async ({ page }) => {
  await page.goto("/");

  // Öffne Session A, produziere viele Zeilen Output
  await createSession(page, "A", "powershell");
  await writeToPty("A", "1..1000 | ForEach-Object { \"Line $_\" }\r\n");
  await expect(page.locator(".xterm-screen")).toContainText("Line 1000");

  // Tab zu Session B, dann zurück zu A
  await createSession(page, "B", "powershell");
  await page.locator('[data-session-tab="A"]').click();

  // Scroll nach oben - sollte "Line 1" finden
  await page.locator(".xterm-viewport").evaluate(el => el.scrollTop = 0);
  await expect(page.locator(".xterm-screen")).toContainText("Line 1");
});
```

Erwartung vor Fix: Assertion-Fail (Line 1 fehlt).

Erwartung nach Fix: Pass.

Risiken & Offene Punkte

Risiko	Mitigation
Memory-Leak bei vielen Sessions	Cache-Eviction in `removeSession`, Store-History mit 200KB-Cap
Hydrate-Flicker beim Remount	Hydrate vor erstem Paint: im selben Synchronous-Effect
Terminal.open() auf neuem DOM-Parent	xterm.js v6 unterstützt Reparent; bei Problemen: term.dispose+recreate, aber mit store-hydrate
ANSI-Escape-Sequenzen im Ring-Buffer	Raw-Bytes speichern (mit Escape-Codes), xterm.write handled sie korrekt
Tests gegen echtes xterm flaky	Playwright mit retries, oder xterm-mock für Unit-Level

Out of Scope für v1.6.26

- Rust-seitiges Backlog (H5) → v1.6.27
- Session-Output persistent auf Disk (Crash-Recovery) → v1.7.0
- Log-Export / Markdown-Save → separate Feature-Diskussion

Timeline & Next Steps

Jetzt (v1.6.26 vor Push)

1. **User reviewed diese Präsentation**
2. Fix-Implementation starten (Phase 1-5, ~3h Gesamtaufwand)
3. Regression-Test schreiben, grün ziehen
4. Full-Gate-Run (tsc, lint, build, vitest, cargo test, clippy)
5. Manuelle Verifikation: Tab-Switch zwischen 2 Sessions mit viel Output
6. Re-Tag v1.6.26 auf Fix-Commit
7. User verifiziert lokal (Release-Verifikations-Regel)
8. Push master + Tag

Post-v1.6.26

- H5 (Rust-Side-Backlog) → v1.6.27
- Design-System-Intake nachziehen (aus backup/origin-master-snapshot)

Zusammenfassung

- **Root-Cause:** `term.dispose()` **bei jedem Tab-Switch zerstört den xterm.js-Scrollback.**
- Aggravator: Kein Store-Ring-Buffer → kein Hydrate möglich beim Remount.
- Aggravator: Pro-Komponenten-Listener → Events in unfokussierten Tabs gehen verloren.

- **Fix: Terminal-Instance-Cache + Store-Ring-Buffer (Fix C, ~120 LOC).**
- **Risk:** Niedrig, gut testbar, Regression-Test + E2E vorhanden.
- **Gewinn:** Scroll-Verlust permanent behoben, Vertrauensaufbau in die App.

Eine Zeile Code, die alles erklärt:

```
}, [sessionId, isAtBottom]);   // SessionTerminal.tsx:184
```

Danke.

Analyse-Artefakte:

- `reports/session-scroll-bug-analyse-v1.6.26.md` (diese Präsentation)
- `reports/origin-master-pre-reconcile.bundle` (Backup von Remote vor Reconcile)
- Backup-Branches: `backup/*` (3 Stück, 4 Wochen aufbewahren)

Nächster Schritt:
Auf User-Entscheidung warten → dann Fix-Implementation starten.

4 Agenten haben diese Analyse produziert:
D1 (Rust-Backend) · D2 (Frontend-Pipeline) · D3 (xterm-Config) · D4 (Git-History)